DevSync

Plan the project, hire a fair team, monitor the work, learn from it

Project Proposal

Course: CSE 3102: Software Design and Development Project
Department of CSE, University of Dhaka

Supervisor: Palash Roy

Team members: Mashfikuzzaman Taeen (Team leader)
Salwa Baki (Backend developer)
Raisa Tabassum (Frontend developer)
Sara Binte Shafayet (Tester)

Date: October 2026

Contents

1 Executive Summary . . . **2**
- 1.1 The problem . . . 2
- 1.2 Our solution . . . 2
- 1.3 Expected outcomes . . . 2

2 Problem Statement . . . **3**
- 2.1 The gaps . . . 3
- 2.2 Why it matters . . . 3

3 Objectives and Goals . . . **4**
- 3.1 Goals . . . 4
- 3.2 Measurable objectives . . . 4

4 Functional Requirements . . . **5**
- 4.1 Use case diagram . . . 5
- 4.2 User roles . . . 6
- 4.3 Features by module . . . 6
- 4.4 User stories . . . 12
- 4.5 Input and output behaviour . . . 13

5 Non-functional Requirements . . . **14**
- 5.1 Performance . . . 14
- 5.2 Security and access control . . . 14
- 5.3 Reliability and availability . . . 14
- 5.4 Scalability . . . 15
- 5.5 Usability and accessibility . . . 15
- 5.6 Maintainability . . . 15

6 Stakeholders . . . **16**
- 6.1 Stakeholders with authority . . . 16
- 6.2 Development team . . . 16
- 6.3 Users . . . 16

7 Timeline and Milestones . . . **17**
- 7.1 Key dates . . . 17
- 7.2 Gantt chart . . . 18

8 Risk Assessment . . . **19**
- 8.1 Risk register . . . 19
- 8.2 Contingency . . . 19

9 Feasibility Study . . . **20**
- 9.1 Technical feasibility . . . 20
- 9.2 Economic feasibility . . . 20
- 9.3 Schedule feasibility . . . 21

10 System Architecture . . . **22**
- 10.1 Architecture diagram . . . 22

References . . . **23**

1. Executive Summary

DevSync is a web platform for running small software projects from start to finish. A client plans the work, gets a fair team of freelance developers, follows the progress and pays one milestone at a time. Whatever happens during the project then goes back into each developer's reputation, so the next team is picked on real results instead of star ratings alone.

1.1 The problem

On most freelancing sites, developers with the most reviews get most of the work, and newcomers find it hard to land even one job. Clients hire people one at a time, plan in one tool and track the work in another. In the end, only a final star rating reaches anyone's reputation, and that rating is easy to fake.

1.2 Our solution

DevSync puts planning, hiring and monitoring in one place. Team selection follows two papers by our course teacher Palash Roy, Prof. Dr. Md. Abdur Razzaque and their co-authors, which use a reputation model and Particle Swarm Optimization (PSO) to form fair software teams. The papers only tested this in simulation, and we are turning it into a working system. On top of that we add Critical Path Method planning, milestone-based monitoring, a minimum number of newcomers that the client chooses for each team, reputation built from evidence with checks against fake reviews, and a plain explanation for every team decision.

1.3 Expected outcomes

- A deployed, tested web application with four roles (client, developer, team lead and admin) that covers the whole project cycle, from planning to payment and reviews.
- A team formation algorithm checked against exact answers on small test cases.
- Simulation results showing how fairly DevSync treats newcomers compared with just hiring the top-rated developers.
- Measurements of how well our review checks stand up to fake and coordinated ratings, along with full project documentation.

2. Problem Statement

Nadia has just finished her CSE degree. She knows React and Node.js well and wants to start freelancing, but every project she bids on goes to someone with hundreds of reviews, and she can't get reviews without a first project. Rahim runs a small business and needs a team for an ordering system. He picks developers one by one, plans in one tool, follows the work in another and has to guess how a late task affects his deadline. At the end he leaves a star rating that says nothing about who was on time or who did most of the work.

Their story points to six gaps in the way software freelancing works today.

2.1 The gaps

Gap	What happens now
New developers are locked out	Platforms rank people by past reviews, so capable newcomers rarely get a first project and never build a record.
Clients get individuals, not teams	Putting together a balanced team with the right skills and budget is left to the client.
Planning, hiring and monitoring are separate	Hiring sites don't plan or track work, and project tools don't form teams, so no one sees the full picture.
Reputation ignores what happened	Missed deadlines, revisions and real contributions never reach a developer's reputation.
Reviews are easy to manipulate	Ratings can be faked, pushed up by friends or used for revenge.
Decisions aren't explained	Developers who lose a job rarely learn why, and clients can't see why one candidate ranked above another.

2.2 Why it matters

Skilled newcomers lose income and often give up before they really start, which hurts in Bangladesh, where many graduates depend on freelancing. Clients pay more, waste time coordinating and often end up with a team that doesn't fit. When reputation is shallow and easy to fake, people stop trusting the platform. Our teachers' research already offers a reputation-aware way to form fair teams, but nobody has built it into a system that real clients and developers can use.

3. Objectives and Goals

We want to build DevSync as a working, tested platform and show with measured results that it forms fair teams without losing much team quality.

3.1 Goals

- Give capable newcomers a fair chance of being selected.
- Bring planning, team formation and monitoring together in one platform.
- Base reputation on what developers actually deliver, and protect it from fake reviews.
- Explain every team decision in plain language.
- Turn the published PCS method into a real system and test it beyond simulation.

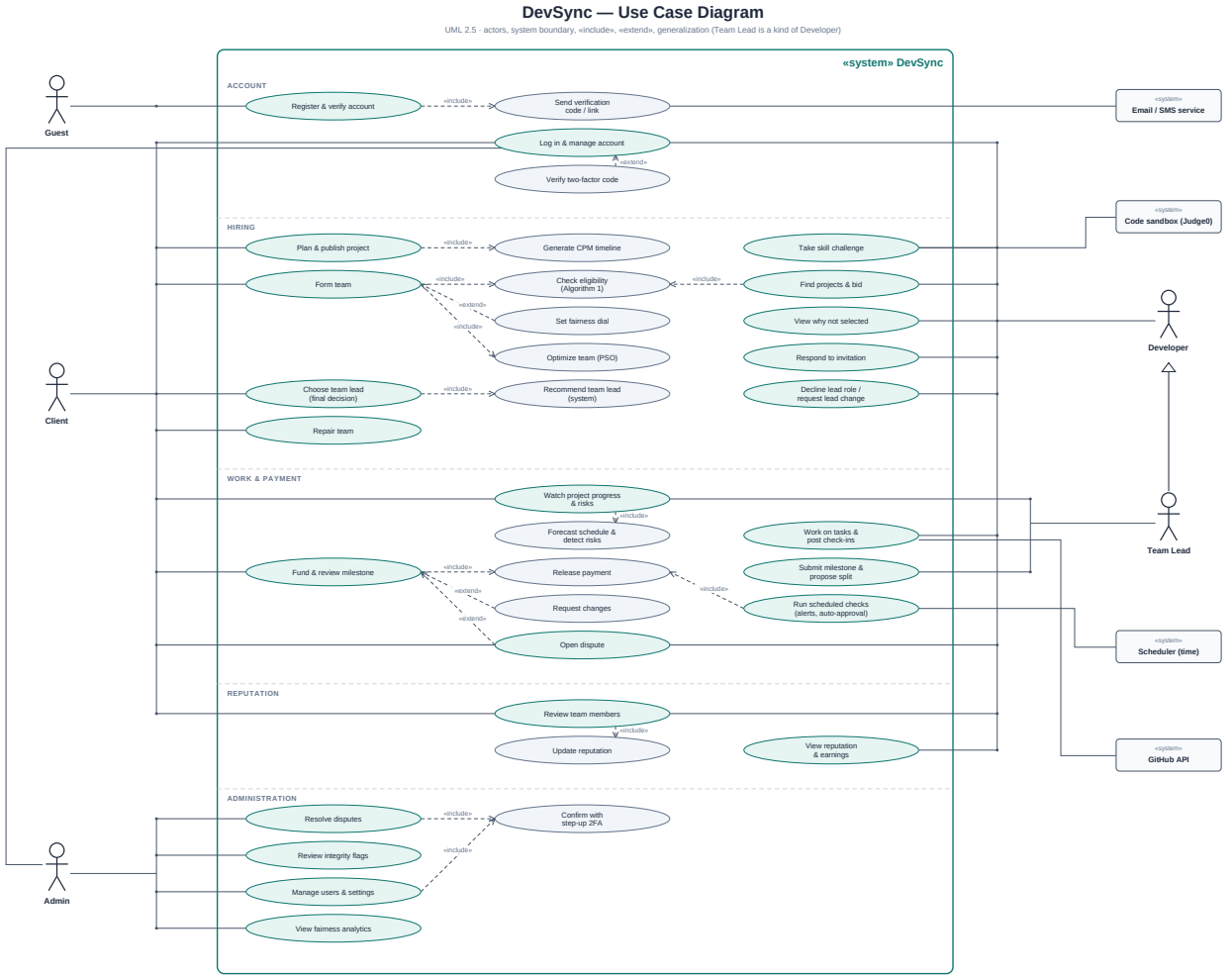
3.2 Measurable objectives

ID	Objective	Target	How we measure it	By
O1	Complete project cycle	Planning, hiring, monitoring, payment, reviews and reputation updates all work end to end	Acceptance and end-to-end tests	7 Dec
O2	Correct project planning	Critical Path Method results match hand-worked answers in every test, and circular dependencies are always rejected	Unit tests with worked examples	2 Nov
O3	Correct team formation	PSO finds the exact best team in at least 95% of small cases (up to 18 bidders), with an average gap of 2% or less	Comparison with an exact solver on 200 cases	7 Dec
O4	Fairer selection	More newcomers selected and work spread more evenly (lower Gini coefficient) than top-rated selection, over 200+ simulated projects	Simulation experiment	4 Jan
O5	Transparent cost of fairness	Shown for every run that uses a newcomer minimum	Feature tests	7 Dec
O6	Security	Permission tests for every endpoint, rate limits on sensitive actions and no high-risk findings in an OWASP ZAP scan	Security tests and scan	4 Jan
O7	Responsiveness	Pages respond within 0.5 seconds (95th percentile) with 50 users at once	Load testing	4 Jan
O8	Testing	Unit tests for every module, with at least 70% coverage overall and 90% for the algorithms	CI coverage report	Every sprint
O9	Usability	A first-time client publishes a 5-milestone project in under 10 minutes without help	Test with 3 classmates	7 Dec
O10	Delivery	Live deployment with full documentation	Final review	4 Jan

These are planning targets. We will report the values we actually measure, whether they come out above or below the target.

4. Functional Requirements

4.1 Use case diagram



Team lead is a kind of developer, so it inherits every developer use case. «include» marks a step that always happens; «extend» marks an optional one.

4.2 User roles

Role	Who	What they can do
Guest	Anyone not logged in	View the landing page and public project listings; sign up
Client	A person or business that needs software built	Plan and publish projects, form and compare teams, choose the team lead, monitor milestones and risks, approve deliverables, fund escrow and release payments, review developers, open disputes
Developer	A freelance developer (sign-up role “Worker”)	Build a profile and portfolio, take skill challenges, find projects and bid, accept invitations, work on tasks, post check-ins, attach evidence, agree to payment splits, review teammates, contest reviews
Team lead	A developer chosen to coordinate one project’s team (not a sign-up role)	Everything a developer can do, plus create and assign tasks, manage the timeline and risks, submit milestones, propose payment splits, for that project only
Admin	Platform operators, created separately	Manage users, resolve disputes, review integrity flags and platform alerts, verify certificates, change settings, view fairness analytics and team-formation runs

Monitoring is shared: the **team lead** manages the day-to-day work inside the team, the **client** oversees milestones, risks, deadlines and payments, and the **system** watches both automatically and alerts whoever needs to act.

4.3 Features by module

M1 — Accounts and authentication (Registration, verification, login, roles, two-factor login for admin)

ID	Requirement	Priority
FR-AUTH-1	Users register with name, email, phone, password and role (Client or Developer). Admin accounts are created separately by another admin.	Core
FR-AUTH-2	Phone numbers are verified with a 6-digit code. Only one account is allowed per phone number.	Core
FR-AUTH-3	Email addresses are verified through a link.	Expected
FR-AUTH-4	Users can reset a forgotten password by email.	Expected
FR-AUTH-5	Admins must use two-factor login and manual verification.	Core
FR-AUTH-6	Users manage settings: password, notification preferences and profile visibility.	Expected

M2 — Profiles, portfolio and skills (Developer and client profiles, portfolio, certificates, availability)

ID	Requirement	Priority
FR-PROF-1	Developers build a profile: bio, skills with self-rated levels, availability, GitHub username and portfolio items (title, description, link, screenshots).	Core
FR-PROF-2	Developers can upload certificates for admin verification; verified certificates are shown with a badge.	Expected
FR-PROF-3	The profile shows reputation (M13), skill-challenge scores, performance metrics, project history, portfolio, certificates, number of different clients, and a newcomer badge where it applies.	Core
FR-PROF-4	Clients have a profile with their organization and project history.	Core
FR-PROF-5	Skills come from a fixed list managed by admins, so they stay consistent.	Core

M3 — Skill challenges (Timed multiple-choice tests and auto-graded coding challenges)

ID	Requirement	Priority
FR-CHAL-1	Timed multiple-choice tests per skill, with a question counter and questions drawn at random from an admin-managed customised question bank.	Core
FR-CHAL-2	After completion the score is shown and added to the profile as verified skill evidence.	Core
FR-CHAL-3	Challenge scores set a newcomer's starting expertise and let newcomers pass filtering for that skill.	Core
FR-CHAL-4	If the coding sandbox is unavailable, an admin can review a newcomer's portfolio instead.	Expected

M4 — Project marketplace (Landing page, project search and filters, project details, recommended projects)

ID	Requirement	Priority
FR-MKT-1	A public landing page explains DevSync (Plan $\rightarrow$ Form team $\rightarrow$ Work $\rightarrow$ Pay $\rightarrow$ Learn), its value (fair team formation, transparent selection, evidence-based reputation, milestone payments, monitoring), and offers "Find work" and "Hire a team".	Core
FR-MKT-2	Developers search open projects and filter by skill, budget, deadline, role, team size and required availability.	Core
FR-MKT-3	Project cards show title, budget, team size, skills and deadline, with "View project" and "Bid" actions.	Core
FR-MKT-4	The project details page shows description, budget, duration, team size, required skills and the milestone list with each milestone's state (completed, current, upcoming).	Core
FR-MKT-5	The developer dashboard recommends open projects matching the developer's skills and availability.	Expected

M5 — Project planning (Multi-step project creation, milestones, dependencies, Critical Path Method timeline)

ID	Requirement	Priority
FR-PROJ-1	Clients create a project through a multi-step form: project information, requirements, required skills and team size, milestones and dependencies, budget and deadline, ending with "Generate project plan".	Core
FR-PROJ-2	Each milestone has a duration, required skills and a share of the payment. The shares must add up to 100%.	Core
FR-PROJ-3	The system calculates the timeline with the Critical Path Method: earliest and latest start and finish, slack, project duration, critical path and critical milestones.	Core
FR-PROJ-4	The plan is shown as a dependency graph and a Gantt chart with the critical path highlighted, plus a table view as an accessible alternative.	Core
FR-PROJ-5	Once the team is confirmed, changing the plan requires agreement from the client and the team lead and creates a new baseline.	Expected
FR-PROJ-6	The original plan can be compared with the current forecast.	Expected

M6 — Bidding and worker filtering (Bids and the papers' worker filtering)

ID	Requirement	Priority
FR-BID-1	Developers place one bid per project: skill slot, total price, days needed (availability), short proposal, and optionally portfolio items and expected contribution.	Core
FR-BID-2	Worker filtering (papers' Algorithm 1): a bid is removed if the worker's performance or competency is below the client's minimum, they cannot finish within the deadline, or their price exceeds the budget. The reason is shown.	Core
FR-BID-3	Newcomer rule (our extension): for newcomers, the competency check uses their skill-challenge score and the performance check is skipped.	Core

M7 — Team formation (PSO team selection, priorities, fairness dial, team comparison, "why this team", "why not selected")

ID	Requirement	Priority
FR-TEAM-1	The team formation dashboard shows the number of bids, eligible workers, optimization progress and the recommended team (member, role, skill level, cost).	Core
FR-TEAM-2	Clients set the importance of competency, teamwork, speed and cost with sliders or presets (Balanced, Budget, Quality, Fast). These are the papers' fitness weights. "Recalculate team" re-runs with new settings.	Core
FR-TEAM-3	The system finds the best team with binary PSO (papers' Algorithm 2), with exactly the required team size, within the budget after the platform fee, and covering all required skills.	Core
FR-TEAM-4	"Why this team?": skill coverage, budget compliance, availability and expected completion date, newcomer inclusion, each member's contribution, and team risks (for example, only one member covers a critical skill, or the team is close to the budget limit).	Core
FR-TEAM-5	Developers who were not selected see why and what would have changed the result, for example a lower price.	Expected
FR-TEAM-6	Every run is saved with its settings so it can be repeated exactly. For up to 18 bidders, an exact solver checks every possible team to verify the PSO result.	Core
FR-TEAM-7	Confirming a team sends invitations to the selected developers.	Core

M8 — Invitations, team lead and team repair (Accepting places, choosing a lead, replacing members)

ID	Requirement	Priority
FR-INV-1	Selected developers accept or decline within 48 hours; no answer counts as declining.	Core
FR-INV-2	When everyone has accepted, the system recommends a team lead (highest teamwork capacity, experience as tie-breaker) and the client makes the final choice.	Core
FR-INV-3	The client can change the lead at any time; members can request a change with a reason; the chosen member can decline the lead role.	Expected
FR-INV-4	Team repair: when a member declines, leaves or becomes unavailable, the screen shows the project impact (forecast delay and affected milestones) and a ranked list of replacement candidates compared by skills, cost and fit. The best is recommended; PSO re-runs with the remaining members fixed. The client accepts a replacement.	Core
FR-INV-5	A replacement inherits unfinished tasks, and payments for remaining milestones are recalculated. If the lead leaves, a new lead is recommended.	Expected

M9 — Team workspace and monitoring (Overview, task board, check-ins, files, evidence, progress, risk dashboard)

ID	Requirement	Priority
FR-MON-1	The team workspace has tabs: Overview, Tasks, Milestones, Team, Check-ins, Files, Payments and Evidence.	Core
FR-MON-2	Kanban task board with To do, In progress, Review and Done columns; tasks can be dragged between columns. The team lead creates and assigns tasks; members move their own.	Core
FR-MON-3	Each task has an assignee, deadline, estimate, difficulty, dependencies on other tasks in the milestone, attached files, comments, evidence links and status.	Core
FR-MON-4	Milestone progress is calculated from task progress.	Core
FR-MON-5	Each member posts a weekly check-in: what they did, what's next and any blockers. No check-in for 7 days triggers an inactivity warning.	Core
FR-MON-6	A shared Files area for the project, with uploads linked to tasks or milestones.	Expected
FR-MON-7	If the project links a GitHub repository, each member's merged pull requests, code reviews and active coding days are shown as supporting evidence.	Expected
FR-MON-8	The monitoring dashboard shows overall progress, schedule progress (planned vs actual), budget usage (released and committed vs budget), forecast finish, deadline impact, team health and an overall risk level.	Core
FR-MON-9	The risk dashboard lists risks by severity (critical, medium, low), each with details: critical-path delay, slack running out, skill dependency (only one member can do remaining critical work), capacity (a member's assigned work exceeds their available time), budget status, stalled tasks and inactive members.	Core
FR-MON-10	The system records activity events (check-in timing, reply times, whether tasks marked done were accepted, whether work started on the promised date) as evidence for reputation.	Core

M10 — Deliverables and milestone review (Submitting milestones, approving or requesting changes)

ID	Requirement	Priority
FR-DEL-1	Members attach evidence (files, links, GitHub pull requests, screenshots) to their tasks.	Expected
FR-DEL-2	The team lead submits each milestone for review with a description, files and the evidence from its tasks.	Core
FR-DEL-3	The client approves the milestone with a 1–5 star rating, or requests changes with a reason and comment. Each request for changes counts as a revision.	Core
FR-DEL-4	The client can open a dispute instead of approving.	Core

M11 — Payments and earnings (simulated) (Escrow, contribution splits, rewards and penalties, earnings)

ID	Requirement	Priority
FR-PAY-1	Every user has a simulated wallet with a full transaction history. Clients can add simulated funds.	Core
FR-PAY-2	The payment dashboard shows each milestone's amount and state: Not funded, Funded (held in escrow), In review, Paid, or Frozen (dispute).	Core
FR-PAY-3	Clients fund each milestone's escrow before work on it starts; the platform fee is recorded at the same time.	Core

ID	Requirement	Priority
FR-PAY-4	The team lead proposes each milestone's contribution split, with a suggestion based on completed tasks and evidence. Members agree or object within 72 hours; objections become disputes. Milestone details show contribution percentages and calculated shares.	Expected
FR-PAY-5	On approval, escrow is released by the split, applying the papers' payment policy: a reward for good quality delivered on time, a penalty for poor quality, otherwise the claimed price.	Core

M12 — Reviews (Verified, blind reviews by clients and teammates)

ID	Requirement	Priority
FR-REV-1	Only participants of a completed project can review: the client reviews each developer, and developers review their teammates. One review per person per project.	Core
FR-REV-2	Each review records the weight it was given and any integrity flags, visible to admins.	Core

M13 — Reputation (the papers' model)

Following the papers, reputation has two main parts, **competency** and **prior performance**, plus a separate **teamwork capacity** used in team formation. Each parameter is a score from 0 to 1, updated after every project, mostly with recent projects counting more (exponentially weighted moving average). The last column shows the plain label used on screen.

Part	Parameter (papers)	Where DevSync gets the data	Shown on screen as
Competency	Expertise and diversity	Completed projects by domain; for newcomers, skill-challenge results	Expertise
Competency	Communication proficiency	On-time check-ins, reply times, whether claims matched results, verification at sign-up, client rating	Communication
Teamwork	Teamwork capacity	Teammates' ratings, helping with blockers, code reviews given	Teamwork
Performance	Project reviews	Client ratings at milestone approval and project end, after integrity checks	Quality
Performance	Re-employment status	Hiring history: many different clients is good, one repeated client is a warning sign	Client diversity
Performance	Timeliness	Actual milestone completion vs deadlines	Punctuality

ID	Requirement	Priority
FR-REP-1	Reputation follows the papers' model: competency (expertise and diversity, communication), teamwork capacity, and performance (project reviews, re-employment status, timeliness).	Core
FR-REP-2	Each parameter is calculated from the sources above, after review-integrity checks.	Core
FR-REP-3	Reputation is updated after every completed project and after dispute decisions, using the papers' moving-average method.	Core

ID	Requirement	Priority
FR-REP-4	The reputation dashboard shows the parameters, competency and performance, their trend across projects, and lets the developer expand each score to see the evidence behind it. Contribution percentages from past projects are shown as history.	Core
FR-REP-5	Weights inside competency and performance are platform settings with the papers' defaults; the calculation sits behind one interface so the exact paper version and agreed adjustments can be switched.	Core
FR-REP-6	Newcomers (fewer than 3 completed projects) start from verifiable evidence (challenge scores, verification) and neutral values for the rest, replaced by real results as they complete projects.	Core
FR-REP-7	Admins can recalculate all reputations from the logs.	Expected

M14 — Disputes (Raising disputes, evidence, admin decisions)

ID	Requirement	Priority
FR-DIS-1	A client or team member can open a dispute about a milestone, a payment split, a review or a member leaving.	Core
FR-DIS-2	The dispute screen shows the disputed milestone and amount, the client's claim, the developer's response, and evidence attached automatically: original requirements, submitted files, revision history, comments and messages, approval history, task board, check-ins and GitHub evidence.	Core
FR-DIS-3	An admin decides: release full payment, partial payment, refund, request a revision, set a new split, or remove a review, always with a written reason.	Core
FR-DIS-4	Payments and reputation updates for the disputed item wait until the dispute is resolved.	Core

M15 — Project completion and post-mortem (Planned vs actual, causes of delays, reputation update summary)

ID	Requirement	Priority
FR-POST-1	When a project finishes, a post-mortem shows planned vs actual duration and budget, completed milestones, critical delays, revisions and team changes.	Expected
FR-POST-2	It explains the causes and impact of major delays, using the forecast history (for example, which critical milestone slipped and by how much).	Expected

M16 — Administration and analytics (Users, flags, disputes, settings, fairness analytics, platform alerts)

ID	Requirement	Priority
FR-ADM-1	The admin dashboard shows users, active and completed projects, active disputes, fairness analytics, worker distribution and platform alerts.	Core
FR-ADM-2	Admins search users, view their activity history, verify certificates, and suspend or restore accounts.	Core
FR-ADM-3	Admins have queues for disputes, integrity flags and platform alerts (suspicious review or bidding patterns, unresolved disputes, recently completed projects).	Core
FR-ADM-4	Admins view all team-formation runs and can check any run against the exact solver.	Expected

ID	Requirement	Priority
FR-ADM-5	Fairness analytics: how often newcomers are selected, how evenly work is spread (Gini coefficient), and a comparison with picking the top-rated developers.	Expected

4.4 User stories

Client

- As a client, I want to break my project into milestones and see the timeline, so I know when it will finish and which parts can't slip.
- As a client, I want to choose whether cost, speed, competency or teamwork matters most, so the team fits my priorities.
- As a client, I want to compare a few candidate teams side by side, so I can see the trade-offs before deciding.
- As a client, I want to see why a team was chosen and what its risks are, so I can trust the recommendation.
- As a client, I want to include a newcomer and see what it costs, so I can support new talent without guessing the risk.
- As a client, I want a risk dashboard that warns me when a delay will push my deadline, so I can act early.
- As a client, I want to approve each milestone before paying for it, so I only pay for work that's done.
- As a client, I want replacement candidates and the impact on my project when someone leaves, so the project doesn't stall.
- As a client, I want a post-mortem at the end, so I understand what went well and what caused delays.

Developer

- As a newcomer, I want to prove my skills through challenges, so I'm not judged on having zero reviews.
- As a developer, I want to know why I wasn't selected and what would change that, so I can improve my next bid.
- As a developer, I want my reputation based on what I actually delivered, and to see the evidence behind each score.
- As a developer, I want my payment to reflect my real contribution, so I'm paid fairly within the team.

Team lead

- As a team lead, I want to split milestones into tasks, assign them and track them on a board, so everyone knows what to do.
- As a team lead, I want to see risks, blockers and each member's progress, so I can help before things fall behind.
- As a team lead, I want to submit a milestone with all the team's evidence, so the client can review it in one place.
- As a team lead, I want to propose a payment split backed by evidence, so the team agrees quickly.

Admin

- As an admin, I want all the evidence attached to a dispute, so I can decide fairly.
- As an admin, I want to see whether newcomers are being selected fairly, so I know the platform is meeting its goal.

4.5 Input and output behaviour

Feature	Input	Output
Register and verify	Name, email, phone, password, role; 6-digit code	Active account, or a clear error
Find projects	Search text and filters (skill, budget, deadline, role, team size, availability)	Matching project cards
Create a project	Information, requirements, skills, team size, milestones, dependencies, budget, deadline	Plan with critical path, slack and finish date, or the milestones in a circular dependency
Place a bid	Skill slot, price, days needed, proposal, optional portfolio and expected contribution	Saved bid, or the reason it was filtered out
Form a team	Priority weights, minimum standards, newcomer minimum	Recommended team, “why this team” with risks, candidate comparison, exact-solver check, cost of fairness; or why no team fits
“Why not selected”	(Automatic)	Reasons, the price at which the developer would have been chosen (if any), the main area to improve
Team repair	Member unavailable	Project impact and ranked replacement candidates
Update a task	Drag to a new column; progress	Updated progress, forecast and risks
Weekly check-in	Done, next, blockers, confidence	Saved check-in; activity evidence updated
Submit a milestone	Description, files, task evidence	Milestone in review
Review a milestone	Approve with stars, request changes with reason, or open a dispute	Payment released, a revision, or a dispute
Contribution split	Percentage per member	Agreed split, or a dispute
Skill challenge	Answers or code	Score and verified skill evidence
Submit a review	Star rating per person	Hidden until reveal; then updated reputation with reasons
Resolve a dispute (admin)	Outcome, amounts or percentages, written reason	Applied outcome, updated wallets and reputation
Post-mortem	(Automatic at completion)	Planned vs actual, causes of delays, reputation changes
Fairness analytics (admin)	Date range or simulation run	Newcomer share, Gini coefficient, comparison with top-rated selection

5. Non-functional Requirements

These requirements say how well DevSync has to work. Each one has a target we can measure and a way to check it.

5.1 Performance

ID	Requirement	How we check it
NFR-PERF-1	Team formation finishes within 5 seconds (95th percentile) for 50 bidders and a team of up to 6. It runs in the background, so the screen never freezes.	Timing tests on the deployed server
NFR-PERF-2	The Critical Path Method calculation takes under 50 ms for projects with up to 200 milestones.	Automated performance test
NFR-PERF-3	Normal pages and actions respond within 0.5 seconds (95th percentile) with 50 users at once.	Load testing
NFR-PERF-4	The first page loads in under 2.5 seconds on a normal broadband connection, and moving between pages takes under 1 second.	Browser performance measurements

5.2 Security and access control

DevSync handles money, reputation and code written by strangers, so security has to cover accounts, permissions, data and abuse.

ID	Requirement	How we check it
NFR-SEC-1	Passwords are hashed with Argon2. Sessions are short-lived (15 minutes, refreshed securely for up to 7 days) and end on logout.	Code review and security tests
NFR-SEC-2	Every action checks the user's role and whether they own or belong to the item. For example, only a project's client can approve its milestones.	Permission test for every endpoint
NFR-SEC-3	Admins must use two-factor login and can't decide disputes or flags that involve themselves.	Security tests
NFR-SEC-4	Protection against the OWASP Top 10, including SQL injection, cross-site scripting, request forgery and server-side request forgery, plus secure browser headers.	OWASP checklist and ZAP scan before each presentation
NFR-SEC-5	Payments and approvals use database locks, so the same payment can't be released twice even if two requests arrive together.	Concurrency tests
NFR-SEC-6	Secrets are kept in environment variables, never in code, and passwords, tokens and codes are never logged. Secret and dependency scans run on every change.	CI checks

5.3 Reliability and availability

ID	Requirement	How we check it
NFR-REL-1	At least 99% uptime during presentations and evaluation.	Uptime monitoring
NFR-REL-2	Payments and state changes either finish completely or don't happen at all.	Transaction tests
NFR-REL-3	If the background worker stops, the rest of the app keeps working, and queued team-formation runs continue once it's back.	Failure tests

ID	Requirement	How we check it
NFR-REL-4	If team formation takes too long, the best valid team found so far is returned and marked as such.	Timeout tests

5.4 Scalability

ID	Requirement	How we check it
NFR-SCALE-1	Web servers keep no session state, so more copies can run behind the load balancer when traffic grows.	Architecture review
NFR-SCALE-2	Heavy work (team formation, reputation updates, integrity checks, GitHub sync) runs in background workers that can be scaled up.	Architecture review
NFR-SCALE-3	Tested with simulated data of at least 500 developers, 200 projects and 5,000 reviews.	Load and data-volume tests
NFR-SCALE-4	Large bidder pools are handled by PSO; the exact solver is only used to check small cases.	Performance tests

5.5 Usability and accessibility

ID	Requirement	How we check it
NFR-USE-1	Every algorithmic decision (team choice, rejection, reputation change, alert) comes with a plain-language explanation.	Feature review
NFR-USE-2	Works on phones and computers.	Testing on several screen sizes
NFR-USE-3	Every screen has clear loading, empty and error states.	Feature review

5.6 Maintainability

ID	Requirement	How we check it
NFR-MAINT-1	Algorithms (reputation, filtering, PSO, Critical Path Method, review checks) sit in a separate module with no database or web code, so they can be tested on their own.	Code review
NFR-MAINT-2	Unit tests for every module, with at least 70% coverage overall and 90% for the algorithms.	CI coverage report
NFR-MAINT-3	Style checks, type checks and all tests must pass before a change is merged, and another team member reviews every change.	CI and pull-request rules
NFR-MAINT-4	Thresholds and weights are settings, not hard-coded numbers.	Code review
NFR-MAINT-5	Documentation is updated with each feature.	Review at each presentation

6. Stakeholders

Stakeholders are the people and groups who take part in DevSync, are affected by it or have a say over it.

6.1 Stakeholders with authority

Stakeholder	Role	What they expect	Influence
Palash Roy (course teacher)	Supervisor and evaluator; co-author of both papers	Correct use of the reputation model and team formation, good engineering, working presentations and honest results	Very high
Prof. Dr. Md. Abdur Razzaque (Chairman, CSE, University of Dhaka)	Co-author of both papers	Faithful use of the research and a high-quality result	Very high
Evaluation panel	Faculty who assess vivas and presentations	A clear design, a working system and members who can explain their part	High

6.2 Development team

Role	Member	Responsibilities
Team leader	Mashfikuzzaman Taeen	Coordination, schedule and scope; research algorithms (reputation model, worker filtering, PSO team formation, Critical Path Method, review integrity); experiments and reports
Backend developer	Salwa Baki	API, database, accounts, projects, bidding, payments, reviews, disputes, integrations
Frontend developer	Raisa Tabassum	All screens, Gantt timeline, dashboards, animations, accessibility
Tester	Sara Binte Shafayet	Test plan, unit and end-to-end test automation, security checks, performance tests, deployment

6.3 Users

Stakeholder	What they want	What worries them
Clients	Easy planning, a good team within budget, clear progress, early delay warnings, safe payments	Paying for poor work; not knowing why a team was picked
Newcomer developers	A fair chance, a way to prove their skills, feedback when not chosen	Being ignored for having no reviews
Experienced developers	A protected track record and pay that matches their contribution	Unfair reviews and payment disputes
Team leads	Tools to plan tasks, follow progress and propose fair splits	Extra coordination work
Administrators	Evidence to settle disputes and spot abuse	Deciding without clear evidence

7. Timeline and Milestones

We work in an Agile, Scrum-based way with twelve one-week sprints, from 12 October 2026 to 3 January 2027. Every sprint ends with tested, working features and a short review. The work is split into three stages, one for each graded presentation.

7.1 Key dates

Date	Milestone	What we deliver
12 Oct 2026	Work starts (Sprint 1)	Repository, tooling and automated tests set up
18 Oct 2026	Proposal and design	This proposal, use case diagram, ERD, architecture and Gantt chart
2 Nov 2026	Demo 1: core platform and planning (Phases 0–5)	Accounts and security, design system and landing page, profiles and skill tests, project planning with the Critical Path Method, marketplace with sealed bids and worker filtering
7 Dec 2026	Demo 2: research engine and full project cycle (Phases 6–12)	Reputation model, PSO team formation with fairness dial and explanations, team lead selection and team repair, workspace and progress monitoring, milestone payments, reviews and reputation updates, disputes and admin console
4 Jan 2027	Demo 3: proof and polish (Phases 13–15)	GitHub evidence and coding challenges, performance and security results, experiments, deployment and final report

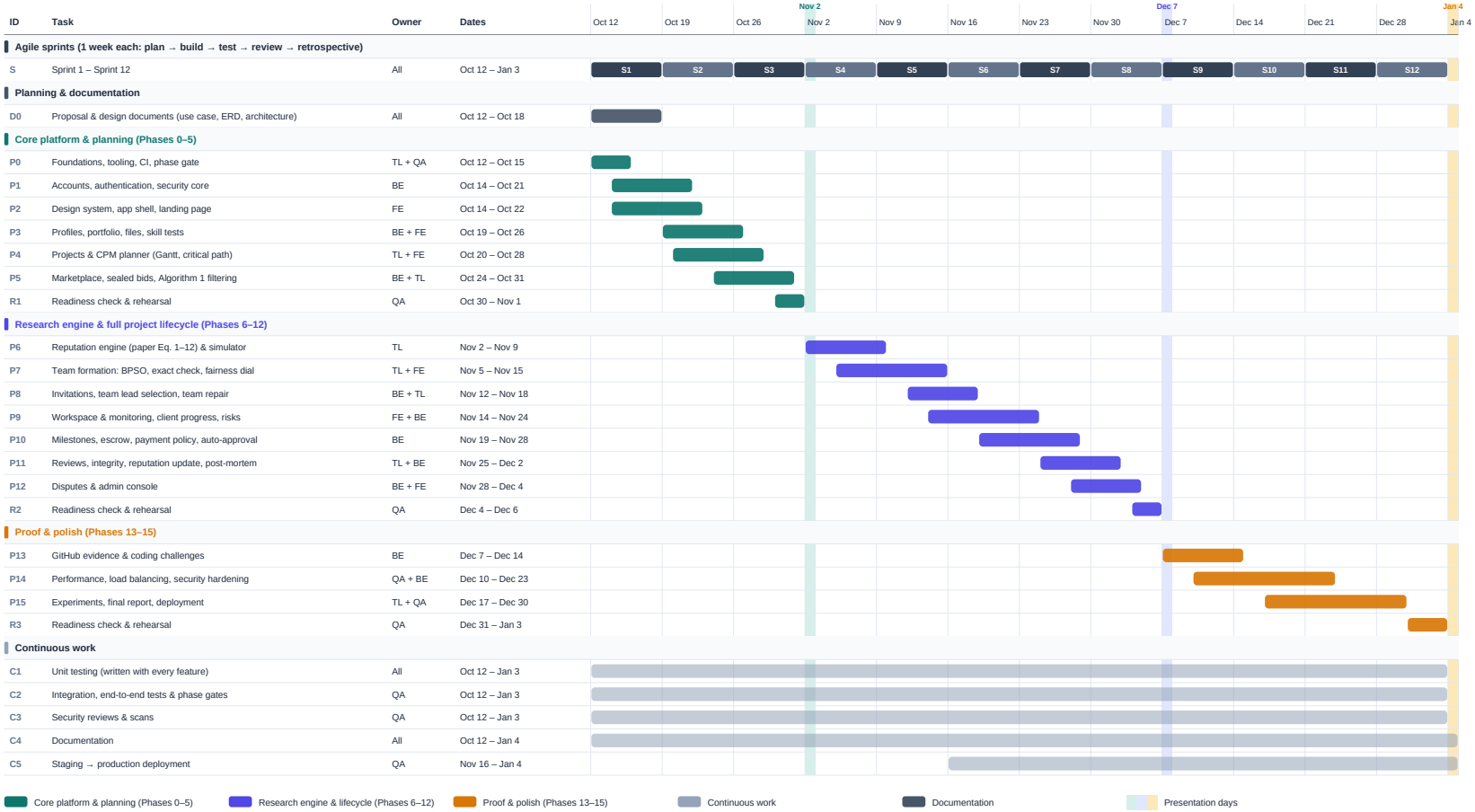
The second stage carries the most work, the first a little less and the third the least. Unit testing, integration testing, security checks and documentation continue through every sprint. The Gantt chart on the next page shows each phase with its dates and owners.

7.2 Gantt chart

DevSync — Project Timeline (Gantt Chart)

12 October 2026 – 4 January 2027 · Agile (Scrum-based) process with one-week sprints · owners: TL = team leader, BE = backend developer, FE = frontend developer, QA = tester

ID	Task	Owner	Dates
	Agile sprints (1 week each: plan → build → test → review → retrospective)		
S	Sprint 1 – Sprint 12	All	Oct 12 – Jan 3
	Planning & documentation		
D0	Proposal & design documents (use case, ERD, architecture)	All	Oct 12 – Oct 18
	Core platform & planning (Phases 0–5)		
P0	Foundations, tooling, CI, phase gate	TL + QA	Oct 12 – Oct 15
P1	Accounts, authentication, security core	BE	Oct 14 – Oct 21
P2	Design system, app shell, landing page	FE	Oct 14 – Oct 22
P3	Profiles, portfolio, files, skill tests	BE + FE	Oct 19 – Oct 26
P4	Projects & CPM planner (Gantt, critical path)	TL + FE	Oct 20 – Oct 28
P5	Marketplace, sealed bids, Algorithm 1 filtering	BE + TL	Oct 24 – Oct 31
R1	Readiness check & rehearsal	QA	Oct 30 – Nov 1
	Research engine & full project lifecycle (Phases 6–12)		
P6	Reputation engine (paper Eq. 1–12) & simulator	TL	Nov 2 – Nov 9
P7	Team formation: BPSO, exact check, fairness dial	TL + FE	Nov 5 – Nov 15
P8	Invitations, team lead selection, team repair	BE + TL	Nov 12 – Nov 18
P9	Workspace & monitoring, client progress, risks	FE + BE	Nov 14 – Nov 24
P10	Milestones, escrow, payment policy, auto-approval	BE	Nov 19 – Nov 28
P11	Reviews, integrity, reputation update, post-mortem	TL + BE	Nov 25 – Dec 2
P12	Disputes & admin console	BE + FE	Nov 28 – Dec 4
R2	Readiness check & rehearsal	QA	Dec 4 – Dec 6
	Proof & polish (Phases 13–15)		
P13	GitHub evidence & coding challenges	BE	Dec 7 – Dec 14
P14	Performance, load balancing, security hardening	QA + BE	Dec 10 – Dec 23
P15	Experiments, final report, deployment	TL + QA	Dec 17 – Dec 30
R3	Readiness check & rehearsal	QA	Dec 31 – Jan 3
	Continuous work		
C1	Unit testing (written with every feature)	All	Oct 12 – Jan 3
C2	Integration, end-to-end tests & phase gates	QA	Oct 12 – Jan 3
C3	Security reviews & scans	QA	Oct 12 – Jan 3
C4	Documentation	All	Oct 12 – Jan 4
C5	Staging → production deployment	QA	Nov 16 – Jan 4



TL = team leader, BE = backend developer, FE = frontend developer, QA = tester. Shaded columns mark the three presentation days.

8. Risk Assessment

Each risk is rated by likelihood and impact (Low, Medium or High), with a plan to reduce it and an owner.

8.1 Risk register

ID	Risk	Likelihood	Impact	Mitigation	Owner
R1	Some formulas in the papers are ambiguous	High	High	Confirm the unclear points with the supervisor early, and keep formulas behind one interface so they can be adjusted	Team leader
R2	New developers have no history, so the model can't score them fairly	High	High	Skill challenges, neutral starting values, the newcomer filtering rule and the fairness dial	Team leader
R3	Fake, colluding or revenge reviews distort reputation	High	High	Verified reviewers only, trust weighting, evidence checks, collusion rules and attack tests	Team leader
R4	Including newcomers costs more team quality than expected	Medium	High	Show the cost of fairness, let clients choose the level, tune the rule and agree the target with the supervisor	Team leader
R5	PSO returns invalid teams	Medium	High	A repair step after every update, random-case tests and comparison with the exact solver	Tester
R6	No real users to test with	High	Medium	Simulated market, demo accounts and usability sessions with classmates	Tester
R7	Deployment problems close to a presentation	Medium	High	Staging deployment from mid-November, health checks and written deployment steps	Tester
R8	A team member becomes unavailable	Low	High	Every module has a second member who reviews and understands it	Team leader
R9	Security weaknesses in money, reviews or admin functions	Medium	High	The security requirements in 5.2, permission tests for every endpoint, rate limits and automated scans	Tester
R10	External services (GitHub, email) fail or hit limits	Medium	Low	Caching, scheduled syncs and fallbacks; these features are optional	Backend developer
R11	Simulated payments seen as unrealistic	Medium	Low	State clearly that money is simulated; the escrow and ledger logic is still real and tested	Backend developer

8.2 Contingency

If we fall behind, we cut features in the order given in 9.3. Core features such as team formation, monitoring, payments, reviews and disputes are never cut.

9. Feasibility Study

This section checks whether we can build DevSync with the skills and tools we have, whether it is worth the effort and whether it fits the schedule.

9.1 Technical feasibility

DevSync uses technologies that are widely taught and libraries with good documentation. The algorithms in the papers are not hard to implement: the reputation model is weighted and moving averages, filtering is a set of threshold checks, and PSO is a well-known algorithm of about 100 to 150 lines. The papers used MATLAB; we will use Python.

Role	Skills needed	Plan for gaps
Team leader	Python, NumPy, PSO, Critical Path Method, basic statistics, project coordination	Implement directly from the papers and check against the exact solver
Backend developer	Python, FastAPI, PostgreSQL, SQLAlchemy, Redis, REST APIs, authentication	Official tutorials in the first sprint; pair with the team leader
Frontend developer	React, TypeScript, Tailwind CSS, charts, accessibility	Start from our working prototype for the Gantt chart and dashboards
Tester	pytest, Playwright, load testing, Docker, GitHub Actions, security testing	Set up CI and the test plan in Sprint 1

- Everything runs on ordinary laptops. No GPU is needed, since DevSync doesn't train any machine-learning models.
- The computation is light. For 50 bidders, 30 particles and 150 iterations, PSO does about 225,000 simple calculations, which takes well under a second.
- Our prototype already runs the reputation model, filtering, PSO, the exact-solver check, the Critical Path Method, payments and review checks. Team formation from 12 bids took well under a second and matched the best of all 495 possible teams.
- All the software is free and open source. Real payments and bank connections are out of reach for a course project, so payments are simulated, and evaluation uses a simulated market, as the papers did.

Conclusion: technically feasible. The main technical risks have mitigations in section 8.

9.2 Economic feasibility

Item	Estimated cost	Notes
Tools and libraries	Free	Open source
Hardware	None	Our own laptops
Domain name (optional)	About BDT 1,000–1,500 a year	A free subdomain also works
SMS, email and code sandbox	Free	Simulated or self-hosted during development
Total	Close to zero	The main cost is our time

The project is worth the effort. It covers the full software engineering cycle, it turns published research into a working system, and fairer access to first projects matters to the many graduates in Bangladesh who rely on freelancing.

Conclusion: economically feasible.

9.3 Schedule feasibility

The plan in section 7 is realistic for four people. Phases overlap, so frontend, backend and algorithm work move forward together, and we start from a working prototype, which removes most of the design uncertainty. Nothing depends on collecting or labelling data. The heaviest stage, from 2 November to 7 December, gets five weeks, and the lighter final stage leaves room for anything that slips.

Every milestone can be measured: each presentation has a fixed list of working features (7.1), and every phase ends with an automated test gate. We review progress at the end of each weekly sprint. If we fall behind, we cut features in this order:

1. Client analytics and notification preferences
2. Email notifications (in-app notifications stay)
3. The separate Files tab (files can still be attached to tasks)
4. GitHub evidence
5. Advanced collusion rules (the basic checks stay)
6. Coding challenges (timed multiple-choice tests stay)
7. Extra team comparison and the price hint in “why not selected”

Any cut is agreed with the supervisor before the presentation and listed as future work in the final report.

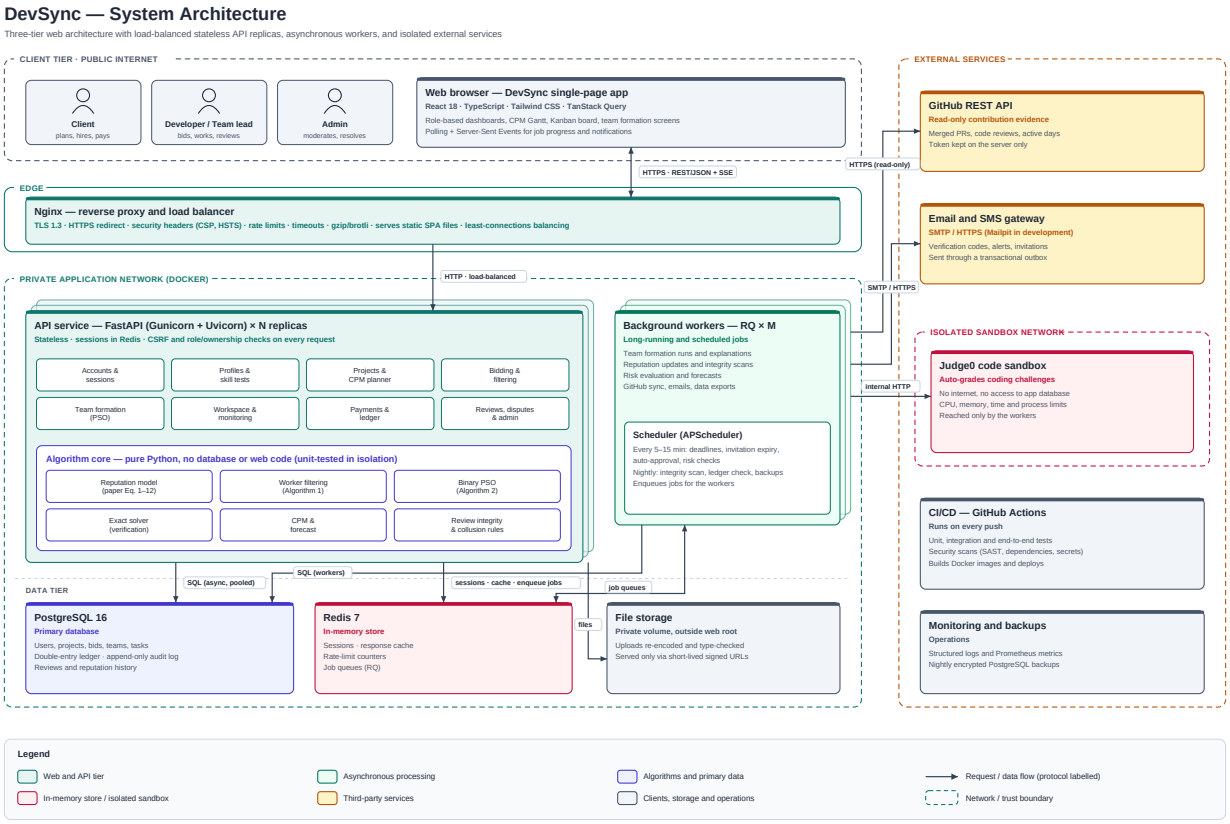
Conclusion: feasible within the semester, with a clear plan if deadlines slip.

10. System Architecture

DevSync uses a three-tier web architecture. The browser runs a React single-page app and talks to the server over HTTPS. Nginx sits at the edge, handles TLS, security headers and rate limits, and spreads requests across several copies of the FastAPI service. These API copies keep no state of their own, so more can be added when traffic grows. Heavy or slow work, such as team formation runs, reputation updates and scheduled checks, goes to background workers through a Redis queue, so the interface never waits on it.

PostgreSQL holds all lasting data, including the payment ledger and an audit log that can't be edited, while Redis keeps sessions, cached responses and rate-limit counters. The algorithms from the papers live in their own module with no database or web code, which lets us test them on their own. Outside services stay at arm's length: GitHub is read-only, email and SMS go through a gateway, and the code sandbox runs on an isolated network with no access to our data.

10.1 Architecture diagram



References

[1] S. T. Rahman, A. K. Sinthia, S. N. Akter, P. Roy and M. A. Razzaque, “Reputation Aware Fair Worker Selection in Collaborative Software Crowdsourcing,” in *Proc. 3rd International Conference on Sustainable Technologies for Industry 4.0 (STI)*, Dhaka, Bangladesh, 2021. Available: https://ieeexplore.ieee.org/document/9732415

[2] S. N. Akter, A. K. Sinthia, P. Roy, M. A. Razzaque, M. M. Hassan, F. Pupo and G. Fortino, “Reputation aware optimal team formation for collaborative software crowdsourcing in industry 5.0,” *Journal of King Saud University – Computer and Information Sciences*, vol. 35, no. 8, p. 101710, 2023. Available: https://doi.org/10.1016/j.jksuci.2023.101710